

# From Gatys to AdaIN

Gram Matrices, Instance Normalization, and the Road to Real-Time Style Transfer

## The two endpoints

Two papers bracket the first era of neural style transfer. Gatys, Ecker, and Bethge opened it in 2015 [1, 2]. Huang and Belongie closed it in 2017 [8]. Between them, style transfer went from minutes per image to milliseconds, and from one picture at a time to any style at all. This report compares the two endpoints, walks the papers that bridged them, and says plainly how AdaIN improves on Gatys.

## Gatys: style as a Gram matrix, found by optimization

Gatys read style off a fixed VGG network. Take the feature maps at layer  $l$ , stored as  $F^l \in \mathbb{R}^{N_l \times M_l}$ , where  $N_l$  is the number of channels and  $M_l$  the spatial size. Style is the correlation between channels, collected in the Gram matrix

$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l, \quad G^l \in \mathbb{R}^{N_l \times N_l}.$$

The style loss is the squared error between the Gram matrices of the generated image and the style image, summed over layers,

$$\mathcal{L}_{\text{style}} = \sum_l w_l \frac{1}{4N_l^2 M_l^2} \|G^l - A^l\|_F^2,$$

with  $A^l$  the style Gram matrix. A content term anchors the layout, and the two are weighted by  $\alpha$  and  $\beta$ .

The important part is how Gatys reaches the target. There is no trained generator. The image itself is the variable. Start from noise, backpropagate the loss to the pixels, and descend with L-BFGS. Every output is a fresh optimization of hundreds of iterations, minutes on a GPU. The method is arbitrary in the styles it accepts, but pays that full price each time [1, 2].

## The stepping stones

Gatys left two problems. It was slow, and each run started from scratch. The field attacked speed first.

Johnson, Alahi, and Fei-Fei replaced the optimization loop with a feed-forward network [3]. They trained an image transformation network against a *perceptual loss* built from the same VGG features and Gram statistics Gatys used, so that a single forward pass produced the stylized image. Ulyanov and colleagues reached the same idea in parallel with Texture Networks, using a multi-scale generator [4]. Both made stylization real-time. Both had the same limit: one network learned one style. A new style meant training a new network.

A separate line pushed on arbitrariness rather than speed. Chen and Schmidt swapped content feature patches for their nearest style patches, an early way to handle an arbitrary style without a per-style model [5], though inverting the features back to an image remained costly.

The decisive turn was normalization. Ulyanov, Vedaldi, and Lempitsky found that swapping batch normalization for *instance normalization* sharply improved feed-forward stylization [6]. Instance normalization standardizes each sample's channels using their own spatial mean and variance,

$$\text{IN}(x) = \gamma \frac{x - \mu(x)}{\sigma(x)} + \beta,$$

where  $\mu(x)$  and  $\sigma(x)$  are computed per channel across spatial positions. The lesson was that per-sample feature statistics carry the style, and removing them normalizes it away.

Dumoulin, Shlens, and Kudlur closed the gap [7]. Their conditional instance normalization let one network render many styles by learning a separate affine pair  $(\gamma^s, \beta^s)$  for each style  $s$ , while sharing every convolution,

$$\text{CIN}(x; s) = \gamma^s \frac{x - \mu(x)}{\sigma(x)} + \beta^s.$$

This showed that the affine parameters of an instance-normalization layer are essentially where the style lives. But the set of styles was still fixed at training time.

### AdaIN: style as channel-wise mean and variance, applied in one pass

Huang and Belongie took the last step [8]. If the affine parameters encode the style, do not learn them. Read them straight off the style image. Given content features  $x$  and style features  $y$ ,

$$\text{AdaIN}(x, y) = \sigma(y) \frac{x - \mu(x)}{\sigma(x)} + \mu(y).$$

Normalize the content channel, then rescale by the style’s standard deviation and shift by the style’s mean. The layer has no learned parameters. Placed between a fixed VGG encoder and a trained decoder, it gives arbitrary style in a single forward pass. It joins the two threads: the speed of the feed-forward networks [3, 4] and the arbitrariness that instance and conditional normalization pointed toward [6, 7].

### Direct comparison

| Aspect               | Gatys [1, 2]                                                          | AdaIN [8]                                              |
|----------------------|-----------------------------------------------------------------------|--------------------------------------------------------|
| Style representation | Gram matrix $G^l = F^l(F^l)^\top$ : all pairwise channel correlations | Per-channel mean $\mu$ and standard deviation $\sigma$ |
| Statistic order      | Full uncentered second order                                          | First order plus diagonal second order                 |
| Numbers per layer    | $N_l^2$                                                               | $2N_l$                                                 |
| How style is applied | Minimized as a loss, by search                                        | Set directly, in closed form                           |
| Speed                | Minutes per image (iterative)                                         | Milliseconds (single forward pass)                     |
| Styles per model     | Any, but re-optimized each time                                       | Any, in one trained model                              |
| Runtime control      | Only $\alpha/\beta$ , before optimizing                               | Content–style knob, style blending, no retraining      |

Three differences carry the weight.

**Gram-matrix style loss (Gatys).** Gatys matches the full  $N_l \times N_l$  Gram matrix, every pairwise channel correlation, as a loss to be minimized [1, 2]. It is a second-order statistic that keeps the cross-channel terms.

**Mean and variance alignment (AdaIN).** AdaIN keeps only the per-channel mean and standard deviation,  $2N_l$  numbers per layer [8]. The link is exact: the diagonal of the Gram matrix,  $G_{ii}^l = \sum_k (F_{ik}^l)^2$ , is the per-channel second moment, from which mean and variance follow. AdaIN keeps that diagonal and drops the off-diagonal correlations. It does not minimize a loss toward the target statistics; it writes them into the features at once. During training, its decoder is supervised by a style loss that matches  $\mu$  and  $\sigma$  across layers, not Gram matrices.

**Iterative optimization to feed-forward transfer.** Gatys optimizes the pixels for every image, minutes at a time [1, 2]. The feed-forward networks removed that loop but bound one

model to one style [3, 4]. AdaIN removes the loop *and* the binding: one trained encoder-decoder handles any style in a single pass [8]. The path is optimization  $\rightarrow$  per-style feed-forward  $\rightarrow$  arbitrary feed-forward.

## How AdaIN improves on Gatys

Speed is the plainest gain. The per-image optimization is gone. Minutes become milliseconds, and the transfer runs in real time.

Generality is the deeper gain. Gatys was arbitrary but slow; the fast networks [3, 4] were quick but fixed to one style. AdaIN is the first method that is both fast and open to any style, because the style enters through statistics computed on the fly rather than through learned weights [8].

Interpretability comes with it. By making the style the affine parameters of a normalization layer, AdaIN explains why instance normalization helped at all [6]: normalization strips per-sample style statistics, and adaptive normalization puts a chosen style’s statistics back. Runtime control follows for free. Interpolating between the content features and the AdaIN output trades content against style without retraining, and mixing several AdaIN outputs blends styles.

The cost is fidelity. AdaIN keeps only per-channel statistics and discards the cross-channel correlations that the Gram matrix holds, so on intricate styles it can miss detail that Gatys, given enough iterations, still captures. AdaIN trades a measure of that faithfulness for speed and for the freedom to stylize anything, at once, in real time.

## References

- [1] L. A. Gatys, A. S. Ecker, and M. Bethge. A Neural Algorithm of Artistic Style. arXiv:1508.06576, 2015.
- [2] L. A. Gatys, A. S. Ecker, and M. Bethge. Image Style Transfer Using Convolutional Neural Networks. CVPR, 2016.
- [3] J. Johnson, A. Alahi, and L. Fei-Fei. Perceptual Losses for Real-Time Style Transfer and Super-Resolution. ECCV, 2016.
- [4] D. Ulyanov, V. Lebedev, A. Vedaldi, and V. Lempitsky. Texture Networks: Feed-forward Synthesis of Textures and Stylized Images. ICML, 2016.
- [5] T. Q. Chen and M. Schmidt. Fast Patch-based Style Transfer of Arbitrary Style. arXiv:1612.04337, 2016.
- [6] D. Ulyanov, A. Vedaldi, and V. Lempitsky. Instance Normalization: The Missing Ingredient for Fast Stylization. arXiv:1607.08022, 2016.
- [7] V. Dumoulin, J. Shlens, and M. Kudlur. A Learned Representation for Artistic Style. ICLR, 2017.
- [8] X. Huang and S. Belongie. Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization. ICCV, 2017.